

Project Phase 4

CMPT 276: Introduction to Software Engineering
Simon Fraser University

Group #15

Megan Chau	Student #301614135
Samantha Gan	Student #301614044
Krish Kaushik	Student #301628888
Tanveer Muntaseer	Student #301620006
Lucas Nguyen	Student #301629229

April 2026

Contents

1	The Game	1
1.1	Game Description	1
1.2	Similarities to Original Plan	1
1.3	Changes from the Original Plan	1
1.4	Lessons Learned	2
2	Tutorial	3
2.1	Video Demo	3

1 The Game

1.1 Game Description

Meowmino's Delivery is a 2D grid-based delivery game built with LibGDX. The player controls a cat riding a delivery bike across three connected maps. The core gameplay is to collect fish scattered across the map, watch and deliver timed orders from houses for fish, and get to the end destination — the White House, all before a 5-minute countdown expires. Players have to avoid enemies in the game: police enemies that patrol the maps and chase on sight (respawning the player with a fish and time penalty if caught), and puddle enemies that slow movement and drains collected fish if the player touches them. If the amount of fish goes below 0, game over! Progress through the three maps is locked behind road barriers: Barrier 1 requires 30 fish; Barrier 2 requires 70 fish. Get past the mob of police to complete the president's delivery!

1.2 Similarities to Original Plan

The following core features from the Phase 1 plan were delivered as designed:

- Map consisting of three sections, with barriers connecting them
- Cat-on-bike player character with WASD movement and directional animation
- Fish collectibles as regular rewards that randomly spawned on map, and pizza deliveries as bonus rewards (+10 per delivery)
- Using police and puddle as moving and non-moving enemies respectively
- Barriers that require a certain condition to be met before clearing
- Five-minute countdown timer ending the game on expiry
- Our team stuck with the assigned roles given (design, front-end, back-end)

1.3 Changes from the Original Plan

Several features were added or modified during development. The table below summarises each change and its justification.

Table 1: Changes from Original Plan

Feature	Status	Justification
Mount/dismount mechanic (Q key; <code>CatOffBike</code> entity)	Added	Adds strategic depth: the player must physically walk to a door, creating a risk-reward trade-off
Off-bike requirement for delivery	Added	Complements mount/dismount mechanic and encourages route planning

Feature	Status	Justification
Obama / White House cutscene ending	Added	Provides a more satisfying narrative conclusion
Barriers clearing requirement	Changed	Switched from delivery count to fish count to allow more flexible progression
Design of screens (StartScreen, Ending Screen)	Changed	Improved visual appeal compared to original mockups
Class designs	Changed	Final implementation diverged from UML due to LibGDX constraints and iterative development
Police penalty	Changed	Now reduces time by 30 seconds in addition to removing fish, increasing difficulty
Iris wipe transition + cinematic letterbox bars	Added	Improves visual polish and transition quality

1.4 Lessons Learned

Communication is essential. Consistent communication ensured that all team members remained aligned on goals, responsibilities, and implementation details, reducing issues and rework.

Game development requires balancing. We learned that implementing even simple gameplay features involves coordinating game logic, rendering, input handling, and testing. Iteration and compromise between design goals and technical limitations were necessary throughout development.

Peer code review reveals blind spots that self-review misses. Assignment 4 code reviews identified concrete smells: duplicated barrier-collision logic in `PoliceEnemy`, data clumps for spawn coordinates in `GameScreen`, an unnecessary `bikeParked` variable mirroring `isOnBike`, and dead code. Addressing this feedback improved both readability and test reliability.

Testing LibGDX entities requires upfront infrastructure planning. Entity constructors load textures and audio, requiring an active OpenGL context. We solved this with Objenesis (to instantiate entities without calling constructors) and Mockito (to stub LibGDX singletons such as `Gdx.graphics` and `Gdx.audio`), but building this infrastructure mid-project was costly. Future projects should design and validate the test harness before writing entity code.

2 Tutorial

2.1 Video Demo

A gameplay demonstration is available at:

<https://youtu.be/gFfb3DsHz78?si=U1MbHLLywqt6zSot>

The video covers all major mechanics: backstory, fish collection, delivery orders, police and puddle enemies, barriers, and the special Obama ending cutscene.

Key	Action
W / A / S / D	Move up / left / down / right
Q	Mount or dismount the bike
E	Deliver an order
ESC	Pause / unpause